

Optimización para Redes Neuronales

Métodos de primer orden y descenso de gradiente

Sergio Hernández
shernandez@ucm.cl



Universidad Católica del Maule

El problema de optimización en deep learning

Entrenar una red es **minimizar** una función de pérdida respecto de los parámetros $\mathbf{x}$ (pesos y sesgos):

$$\min_{\mathbf{x}} f(\mathbf{x}), \quad f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \ell(\hat{y}_i(\mathbf{x}), y_i).$$

El problema de optimización en deep learning

Entrenar una red es **minimizar** una función de pérdida respecto de los parámetros $\mathbf{x}$ (pesos y sesgos):

$$\min_{\mathbf{x}} f(\mathbf{x}), \quad f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \ell(\hat{y}_i(\mathbf{x}), y_i).$$

Dos objetivos que no coinciden

La **optimización** busca minimizar el error sobre los datos de *entrenamiento*; el **aprendizaje** busca minimizar el error de *generalización*. Reducir uno no garantiza reducir el otro.

El problema de optimización en deep learning

Entrenar una red es **minimizar** una función de pérdida respecto de los parámetros $\mathbf{x}$ (pesos y sesgos):

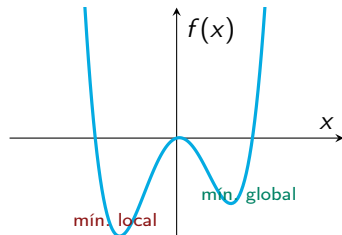
$$\min_{\mathbf{x}} f(\mathbf{x}), \quad f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \ell(\hat{y}_i(\mathbf{x}), y_i).$$

Dos objetivos que no coinciden

La **optimización** busca minimizar el error sobre los datos de *entrenamiento*; el **aprendizaje** busca minimizar el error de *generalización*. Reducir uno no garantiza reducir el otro.

- ▶ En general f es **no convexa** y de muy alta dimensión (millones de parámetros).
- ▶ No buscamos el óptimo global exacto: basta una solución suficientemente buena, obtenida rápido.

- ▶ **Mínimos locales:** un punto mejor que sus vecinos, pero no el mejor global.
- ▶ **Puntos de silla:** gradiente nulo que no es ni máximo ni mínimo; abundan en alta dimensión.
- ▶ **Regiones planas:** gradiente diminuto
⇒ avance lento (gradientes que se desvanecen).



En la práctica, los métodos de primer orden funcionan sorprendentemente bien pese a estos obstáculos: muchos mínimos locales en redes grandes tienen pérdidas similares.

Función convexa

f es convexa si para todo $\lambda \in [0, 1]$: $f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2)$. El segmento entre dos puntos del grafo nunca queda por debajo de la curva.

- ▶ En problemas convexos, *todo* mínimo local es global: la optimización es “fácil”.

Función convexa

f es convexa si para todo $\lambda \in [0, 1]$: $f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2)$. El segmento entre dos puntos del grafo nunca queda por debajo de la curva.

- ▶ En problemas convexos, *todo* mínimo local es global: la optimización es “fácil”.
- ▶ Las redes neuronales son **no convexas**, pero la intuición convexa guía el diseño de los algoritmos.

Función convexa

f es convexa si para todo $\lambda \in [0, 1]$: $f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2)$. El segmento entre dos puntos del grafo nunca queda por debajo de la curva.

- ▶ En problemas convexos, *todo* mínimo local es global: la optimización es “fácil”.
- ▶ Las redes neuronales son **no convexas**, pero la intuición convexa guía el diseño de los algoritmos.
- ▶ Herramientas clave que reutilizaremos: la **serie de Taylor de primer orden** y el **gradiente**.

Descenso de gradiente: la dirección de máximo descenso

Definimos el gradiente en la iteración k como $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$. La aproximación de Taylor de primer orden de un paso de tamaño α en dirección $\mathbf{d}$ es:

$$f(\mathbf{x}^{(k)} + \alpha \mathbf{d}) \approx f(\mathbf{x}^{(k)}) + \alpha \mathbf{d}^\top \mathbf{g}^{(k)}.$$

Descenso de gradiente: la dirección de máximo descenso

Definimos el gradiente en la iteración k como $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$. La aproximación de Taylor de primer orden de un paso de tamaño α en dirección $\mathbf{d}$ es:

$$f(\mathbf{x}^{(k)} + \alpha \mathbf{d}) \approx f(\mathbf{x}^{(k)}) + \alpha \mathbf{d}^\top \mathbf{g}^{(k)}.$$

- Para reducir f tanto como sea posible (con $\|\mathbf{d}\| = 1$), elegimos la dirección **opuesta al gradiente**:

$$\mathbf{d}^{(k)} = -\frac{\mathbf{g}^{(k)}}{\|\mathbf{g}^{(k)}\|}.$$

Descenso de gradiente: la dirección de máximo descenso

Definimos el gradiente en la iteración k como $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$. La aproximación de Taylor de primer orden de un paso de tamaño α en dirección $\mathbf{d}$ es:

$$f(\mathbf{x}^{(k)} + \alpha \mathbf{d}) \approx f(\mathbf{x}^{(k)}) + \alpha \mathbf{d}^\top \mathbf{g}^{(k)}.$$

- ▶ Para reducir f tanto como sea posible (con $\|\mathbf{d}\| = 1$), elegimos la dirección **opuesta al gradiente**:

$$\mathbf{d}^{(k)} = -\frac{\mathbf{g}^{(k)}}{\|\mathbf{g}^{(k)}\|}.$$

- ▶ Siguiendo esa dirección, con α suficientemente pequeño y gradiente no nulo, el avance *garantiza* mejora.
- ▶ Un punto con gradiente cero es un **punto estacionario**.

Algoritmo de descenso de gradiente

- 1: **Entrada:** función f , gradiente ∇f , punto inicial $\mathbf{x}^{(1)}$, tasa α .
- 2: **for** $k = 1, 2, \dots$ hasta convergencia **do**
- 3: Calcular el gradiente $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$.
- 4: Actualizar $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \mathbf{g}^{(k)}$.
- 5: **end for**
- 6: **Devolver** $\mathbf{x}^{(k)}$.

- ▶ La **tasa de aprendizaje** α es el hiperparámetro más delicado:
 - ▶ demasiado pequeña $\Rightarrow$ convergencia lentísima;
 - ▶ demasiado grande $\Rightarrow$ oscilaciones o divergencia.

Algoritmo de descenso de gradiente

- 1: **Entrada:** función f , gradiente ∇f , punto inicial $\mathbf{x}^{(1)}$, tasa α .
- 2: **for** $k = 1, 2, \dots$ hasta convergencia **do**
- 3: Calcular el gradiente $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$.
- 4: Actualizar $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \mathbf{g}^{(k)}$.
- 5: **end for**
- 6: **Devolver** $\mathbf{x}^{(k)}$.

- ▶ La **tasa de aprendizaje** α es el hiperparámetro más delicado:
 - ▶ demasiado pequeña $\Rightarrow$ convergencia lentísima;
 - ▶ demasiado grande $\Rightarrow$ oscilaciones o divergencia.
- ▶ Sin normalizar la dirección, α no corresponde a la longitud real del paso.

- ▶ Si optimizamos el paso en cada iteración (*line search* exacta), la nueva dirección resulta **ortogonal** a la anterior:

$$\mathbf{d}^{(k+1)\top} \mathbf{d}^{(k)} = 0.$$

- ▶ Si optimizamos el paso en cada iteración (*line search* exacta), la nueva dirección resulta **ortogonal** a la anterior:

$$\mathbf{d}^{(k+1)\top} \mathbf{d}^{(k)} = 0.$$

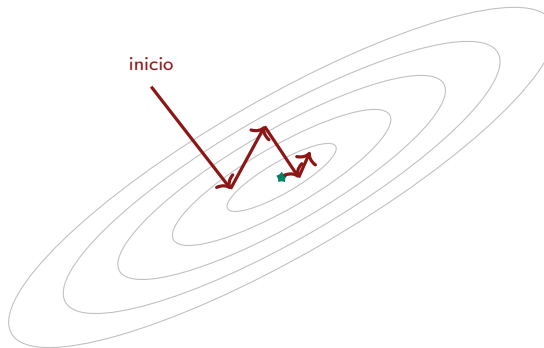
- ▶ Esto produce trayectorias en **zig-zag** en valles alargados (p. ej. la función de Rosenbrock).

El problema de los valles estrechos

- ▶ Si optimizamos el paso en cada iteración (*line search* exacta), la nueva dirección resulta **ortogonal** a la anterior:

$$\mathbf{d}^{(k+1)\top} \mathbf{d}^{(k)} = 0.$$

- ▶ Esto produce trayectorias en **zig-zag** en valles alargados (p. ej. la función de Rosenbrock).
- ▶ El descenso de gradiente puro es sensible al **escalamiento** de las variables.



Curvas de nivel de un valle estrecho y el zig-zag del descenso de gradiente.

Descenso de gradiente estocástico (SGD)

- ▶ El gradiente completo $\nabla f(\mathbf{x}) = \frac{1}{n} \sum_i \nabla \ell_i(\mathbf{x})$ cuesta $O(n)$ por paso: inviable con millones de ejemplos.

Descenso de gradiente estocástico (SGD)

- ▶ El gradiente completo $\nabla f(\mathbf{x}) = \frac{1}{n} \sum_i \nabla \ell_i(\mathbf{x})$ cuesta $O(n)$ por paso: inviable con millones de ejemplos.
- ▶ **SGD**: en cada paso se toma un único ejemplo i al azar y se usa su gradiente como *estimador insesgado*:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \nabla \ell_i(\mathbf{x}^{(k)}), \quad \mathbb{E}[\nabla \ell_i(\mathbf{x})] = \nabla f(\mathbf{x}).$$

Descenso de gradiente estocástico (SGD)

- ▶ El gradiente completo $\nabla f(\mathbf{x}) = \frac{1}{n} \sum_i \nabla \ell_i(\mathbf{x})$ cuesta $O(n)$ por paso: inviable con millones de ejemplos.
- ▶ **SGD**: en cada paso se toma un único ejemplo i al azar y se usa su gradiente como *estimador insesgado*:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \nabla \ell_i(\mathbf{x}^{(k)}), \quad \mathbb{E}[\nabla \ell_i(\mathbf{x})] = \nabla f(\mathbf{x}).$$

- ▶ Cada paso es n veces más barato, pero el gradiente es **ruidoso**: la trayectoria fluctúa.

Descenso de gradiente estocástico (SGD)

- ▶ El gradiente completo $\nabla f(\mathbf{x}) = \frac{1}{n} \sum_i \nabla \ell_i(\mathbf{x})$ cuesta $O(n)$ por paso: inviable con millones de ejemplos.
- ▶ **SGD**: en cada paso se toma un único ejemplo i al azar y se usa su gradiente como *estimador insesgado*:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \nabla \ell_i(\mathbf{x}^{(k)}), \quad \mathbb{E}[\nabla \ell_i(\mathbf{x})] = \nabla f(\mathbf{x}).$$

- ▶ Cada paso es n veces más barato, pero el gradiente es **ruidoso**: la trayectoria fluctúa.
- ▶ El ruido puede *ayudar* a escapar de mínimos locales pobres y puntos de silla.
- ▶ Conviene **reducir** α con el tiempo para amortiguar el ruido cerca del óptimo.

Minibatch SGD: el compromiso práctico

- Un punto intermedio entre el gradiente completo y SGD puro: usar un **minibatch** $\mathcal{B}$ de $|\mathcal{B}| = b$ ejemplos.

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \frac{\alpha}{b} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{x}^{(k)}).$$

- ▶ Un punto intermedio entre el gradiente completo y SGD puro: usar un **minibatch** $\mathcal{B}$ de $|\mathcal{B}| = b$ ejemplos.

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \frac{\alpha}{b} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{x}^{(k)}).$$

- ▶ **Ventajas:**
 - ▶ Reduce la varianza del gradiente por un factor $\approx 1/b$.
 - ▶ Aprovecha el paralelismo del hardware (GPU): multiplicaciones matriciales eficientes.

- ▶ Un punto intermedio entre el gradiente completo y SGD puro: usar un **minibatch** $\mathcal{B}$ de $|\mathcal{B}| = b$ ejemplos.

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \frac{\alpha}{b} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{x}^{(k)}).$$

- ▶ **Ventajas:**
 - ▶ Reduce la varianza del gradiente por un factor $\approx 1/b$.
 - ▶ Aprovecha el paralelismo del hardware (GPU): multiplicaciones matriciales eficientes.
- ▶ El tamaño de lote b es un compromiso entre *eficiencia computacional* y *calidad del gradiente*; valores típicos: 32 a 512.
- ▶ Es el caballo de batalla del entrenamiento moderno; todo lo que sigue se construye sobre él.

Gradiente conjugado: superar los valles

Inspirado en la optimización de funciones cuadráticas $\frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$, con $\mathbf{A}$ simétrica y definida positiva.

- Genera direcciones **mutuamente conjugadas** respecto de $\mathbf{A}$:

$$\mathbf{d}^{(i)\top} \mathbf{A} \mathbf{d}^{(j)} = 0 \quad \text{para todo } i \neq j.$$

Gradiente conjugado: superar los valles

Inspirado en la optimización de funciones cuadráticas $\frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$, con $\mathbf{A}$ simétrica y definida positiva.

- ▶ Genera direcciones **mutuamente conjugadas** respecto de $\mathbf{A}$:

$$\mathbf{d}^{(i)\top} \mathbf{A} \mathbf{d}^{(j)} = 0 \quad \text{para todo } i \neq j.$$

- ▶ Converge en a lo sumo n pasos en un problema cuadrático de dimensión n .

Gradiente conjugado: superar los valles

Inspirado en la optimización de funciones cuadráticas $\frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$, con $\mathbf{A}$ simétrica y definida positiva.

- ▶ Genera direcciones **mutuamente conjugadas** respecto de $\mathbf{A}$:

$$\mathbf{d}^{(i)\top} \mathbf{A} \mathbf{d}^{(j)} = 0 \quad \text{para todo } i \neq j.$$

- ▶ Converge en a lo sumo n pasos en un problema cuadrático de dimensión n .
- ▶ Cada dirección combina el gradiente actual con la dirección anterior:

$$\mathbf{d}^{(k)} = -\mathbf{g}^{(k)} + \beta^{(k)} \mathbf{d}^{(k-1)}.$$

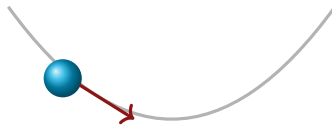
un $\beta^{(k)}$ mayor da más peso a la dirección previa.

Momentum: acumular inercia

- ▶ En regiones casi planas, el gradiente es pequeño y el avance es lento.
- ▶ **Momentum** acumula una *velocidad* $\mathbf{v}$ que promedia los gradientes recientes:

$$\begin{aligned}\mathbf{v}^{(k+1)} &= \beta \mathbf{v}^{(k)} - \alpha \mathbf{g}^{(k)}, \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}.\end{aligned}$$

- ▶ Con $\beta = 0$ se recupera el descenso de gradiente.



la “bola” acumula inercia

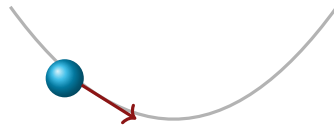
Interpretación física: una bola que rueda cuesta abajo gana velocidad; el gradiente actúa como la gravedad.

Momentum: acumular inercia

- ▶ En regiones casi planas, el gradiente es pequeño y el avance es lento.
- ▶ **Momentum** acumula una *velocidad* $\mathbf{v}$ que promedia los gradientes recientes:

$$\begin{aligned}\mathbf{v}^{(k+1)} &= \beta \mathbf{v}^{(k)} - \alpha \mathbf{g}^{(k)}, \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}.\end{aligned}$$

- ▶ Con $\beta = 0$ se recupera el descenso de gradiente.



la "bola" acumula inercia

Interpretación física: una bola que rueda cuesta abajo gana velocidad; el gradiente actúa como la gravedad.

El momentum suaviza el zig-zag: cancela las componentes oscilatorias y refuerza la dirección consistente hacia el valle.

- ▶ Problema del momentum clásico: no frena lo suficiente al fondo del valle y tiende a *sobrepasarlo*.

- ▶ Problema del momentum clásico: no frena lo suficiente al fondo del valle y tiende a *sobrepasarlo*.
- ▶ **Nesterov**: evaluar el gradiente en la **posición futura proyectada** $\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}$, no en la actual.

$$\begin{aligned}\mathbf{v}^{(k+1)} &= \beta \mathbf{v}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}), \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}.\end{aligned}$$

- ▶ Problema del momentum clásico: no frena lo suficiente al fondo del valle y tiende a *sobrepasarlo*.
- ▶ **Nesterov**: evaluar el gradiente en la **posición futura proyectada** $\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}$, no en la actual.

$$\begin{aligned}\mathbf{v}^{(k+1)} &= \beta \mathbf{v}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}), \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}.\end{aligned}$$

- ▶ Es una *corrección anticipada*: “mira hacia dónde voy” antes de calcular el ajuste.
- ▶ En la práctica reduce el sobrepaso y suele converger algo más rápido que el momentum clásico.

AdaGrad: una tasa por parámetro

Momentum aplica la misma tasa a todas las componentes. **AdaGrad** adapta una tasa para *cada* parámetro x_i según su historial de gradientes.

$$s_i^{(k)} = \sum_{j=1}^k (g_i^{(j)})^2, \quad x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}} g_i^{(k)}.$$

- ▶ Parámetros con gradientes grandes reciben pasos más pequeños, y viceversa: útil con gradientes **dispersos**.
- ▶ Poco sensible a la tasa base (a menudo $\alpha = 0.01$); $\epsilon \sim 10^{-8}$ evita dividir por cero.

AdaGrad: una tasa por parámetro

Momentum aplica la misma tasa a todas las componentes. **AdaGrad** adapta una tasa para *cada* parámetro x_i según su historial de gradientes.

$$s_i^{(k)} = \sum_{j=1}^k (g_i^{(j)})^2, \quad x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}} g_i^{(k)}.$$

- ▶ Parámetros con gradientes grandes reciben pasos más pequeños, y viceversa: útil con gradientes **dispersos**.
- ▶ Poco sensible a la tasa base (a menudo $\alpha = 0.01$); $\epsilon \sim 10^{-8}$ evita dividir por cero.
- ▶ **Debilidad**: s_i sólo crece $\Rightarrow$ la tasa efectiva decae monótonamente y puede volverse *infinitesimal* antes de converger.

RMSProp

- Reemplaza la suma acumulada por una **media móvil** con decaimiento $\gamma \approx 0.9$:

$$\hat{s}^{(k+1)} = \gamma \hat{s}^{(k)} + (1 - \gamma)(\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)})$$

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{\hat{s}_i^{(k+1)}}} g_i^{(k)}$$

La tasa efectiva ya no decae sin límite.

$\odot$ es el producto elemento a elemento; RMS denota la raíz cuadrática media con decaimiento.

Adadelta

- También evita el decaimiento monótono, y además **elimina** la tasa α .
- Usa una media móvil de los *cuadrados de las actualizaciones* para corregir las unidades:

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\text{RMS}(\Delta x_i)}{\epsilon + \text{RMS}(g_i)} g_i^{(k)}$$

Adam combina un promedio decreciente del gradiente (como momentum) y de su cuadrado (como RMSProp), con *corrección de sesgo* inicial.

$$\mathbf{v}^{(k+1)} = \gamma_v \mathbf{v}^{(k)} + (1 - \gamma_v) \mathbf{g}^{(k)} \quad (1^{\text{er}} \text{ momento})$$

$$\mathbf{s}^{(k+1)} = \gamma_s \mathbf{s}^{(k)} + (1 - \gamma_s) (\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}) \quad (2^{\text{do}} \text{ momento})$$

$$\hat{\mathbf{v}}^{(k+1)} = \mathbf{v}^{(k+1)} / (1 - \gamma_v^k), \quad \hat{\mathbf{s}}^{(k+1)} = \mathbf{s}^{(k+1)} / (1 - \gamma_s^k) \quad (\text{corrección})$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \hat{\mathbf{v}}^{(k+1)} / (\epsilon + \sqrt{\hat{\mathbf{s}}^{(k+1)}}) \quad (\text{actualización})$$

Adam: momentum + tasa adaptativa

Adam combina un promedio decreciente del gradiente (como momentum) y de su cuadrado (como RMSProp), con *corrección de sesgo* inicial.

$$\mathbf{v}^{(k+1)} = \gamma_v \mathbf{v}^{(k)} + (1 - \gamma_v) \mathbf{g}^{(k)} \quad (1^{\text{er}} \text{ momento})$$

$$\mathbf{s}^{(k+1)} = \gamma_s \mathbf{s}^{(k)} + (1 - \gamma_s) (\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}) \quad (2^{\text{do}} \text{ momento})$$

$$\hat{\mathbf{v}}^{(k+1)} = \mathbf{v}^{(k+1)} / (1 - \gamma_v^k), \quad \hat{\mathbf{s}}^{(k+1)} = \mathbf{s}^{(k+1)} / (1 - \gamma_s^k) \quad (\text{corrección})$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \hat{\mathbf{v}}^{(k+1)} / (\epsilon + \sqrt{\hat{\mathbf{s}}^{(k+1)}}) \quad (\text{actualización})$$

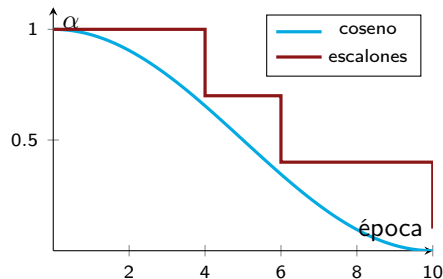
Valores por defecto (paper original)

$\alpha = 0.001$, $\gamma_v = 0.9$, $\gamma_s = 0.999$, $\epsilon = 10^{-8}$. Es el optimizador más usado por defecto en deep learning.

Método	Idea añadida	Hiperparámetros
SGD	Gradiente ruidoso por minibatch	α
Momentum	Inercia / velocidad acumulada	α, β
Nesterov	Gradiente en posición proyectada	α, β
AdaGrad	Tasa por parámetro (suma de g^2)	α
RMSProp	Media móvil de g^2	α, γ
Adadelta	RMSProp sin tasa α	γ_s, γ_x
Adam	Momentum + RMSProp + corrección	$\alpha, \gamma_v, \gamma_s$

- **Punto de partida razonable:** Adam (rápido y robusto) o SGD con momentum (suele generalizar muy bien con buen ajuste).

- ▶ Mantener α fija rara vez es óptimo: conviene **variarla** durante el entrenamiento.
- ▶ Esquemas comunes:
 - ▶ *Decaimiento por escalones.*
 - ▶ *Decaimiento coseno.*
 - ▶ *Calentamiento (warmup) inicial.*
- ▶ Empezar alto (avanzar rápido) y bajar (asentarse en el mínimo) suele dar lo mejor de ambos mundos.



- ▶ Los métodos acelerados son muy sensibles a la tasa α , o gastan mucho esfuerzo en adaptarla.

- ▶ Los métodos acelerados son muy sensibles a la tasa α , o gastan mucho esfuerzo en adaptarla.
- ▶ Idea: aplicar *descenso de gradiente a la propia tasa*. Un **hipergradiente** es la derivada de f respecto de un hiperparámetro.

- ▶ Los métodos acelerados son muy sensibles a la tasa α , o gastan mucho esfuerzo en adaptarla.
- ▶ Idea: aplicar *descenso de gradiente a la propia tasa*. Un **hipergradiente** es la derivada de f respecto de un hiperparámetro.
- ▶ Para el descenso de gradiente, la derivada respecto de α resulta:

$$\frac{\partial f(\mathbf{x}^{(k+1)})}{\partial \alpha^{(k)}} = \mathbf{g}^{(k+1)\top} (-\mathbf{g}^{(k)}), \quad \alpha^{(k)} = \alpha^{(k-1)} + \mu \mathbf{g}^{(k)\top} \mathbf{g}^{(k-1)}.$$

- ▶ Los métodos acelerados son muy sensibles a la tasa α , o gastan mucho esfuerzo en adaptarla.
- ▶ Idea: aplicar *descenso de gradiente a la propia tasa*. Un **hipergradiente** es la derivada de f respecto de un hiperparámetro.
- ▶ Para el descenso de gradiente, la derivada respecto de α resulta:

$$\frac{\partial f(\mathbf{x}^{(k+1)})}{\partial \alpha^{(k)}} = \mathbf{g}^{(k+1)\top} (-\mathbf{g}^{(k)}), \quad \alpha^{(k)} = \alpha^{(k-1)} + \mu \mathbf{g}^{(k)\top} \mathbf{g}^{(k-1)}.$$

- ▶ Sólo requiere recordar el gradiente anterior; reduce la sensibilidad a la elección inicial de α .

- ▶ El **descenso de gradiente** sigue la dirección de máximo descenso; su tasa de aprendizaje es crítica.
- ▶ **SGD / minibatch** hacen viable el entrenamiento a gran escala a costa de gradientes ruidosos.
- ▶ El **gradiente conjugado** se adapta a los valles locales reutilizando información previa.
- ▶ El **momentum** (y Nesterov) acumula progreso en direcciones favorables y suaviza el zig-zag.
- ▶ Los métodos de **tasa adaptativa** (AdaGrad, RMSProp, Adadelta, **Adam**) ajustan el paso por parámetro.
- ▶ La **programación de la tasa** y el **hipergradiente** reducen la dependencia del ajuste manual.

No existe un optimizador único “mejor”: la elección depende del problema, los datos y el presupuesto de cómputo.

- ▶ Zhang, Lipton, Li & Smola, *Dive into Deep Learning*, cap. 12 (Optimization Algorithms) https://d2l.ai/chapter_optimization/index.html
- ▶ Kochenderfer & Wheeler, *Algorithms for Optimization*, cap. 5 (First-Order Methods), MIT Press.
- ▶ Goodfellow, Bengio & Courville, *Deep Learning*, cap. 8 (Optimization for Training Deep Models).
- ▶ Kingma & Ba (2015), *Adam: A Method for Stochastic Optimization*, ICLR.
- ▶ Duchi, Hazan & Singer (2011), *Adaptive Subgradient Methods* (AdaGrad), JMLR.
- ▶ Nesterov (1983); Baydin et al. (2018), *Online Learning Rate Adaptation with Hypergradient Descent*.